

Programming Assignment 2: HTTP Web Proxy Server

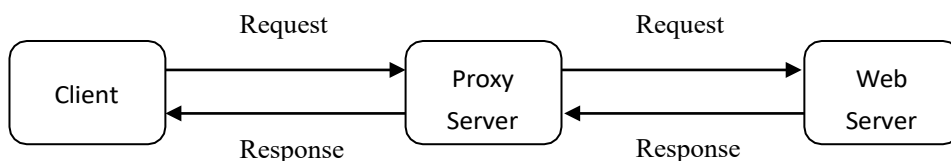
Assigned 3/3

Due: 3/27

In this assignment, you will learn how web proxy servers work and one of their basic functionalities – caching.

Your task is to develop a small web proxy server which is able to cache web pages. It is a very simple proxy server which only understands simple GET-requests, but is able to handle all kinds of objects - not just HTML pages, but also images.

Generally, when the client makes a request, the request is sent to the original web server. The original web server then processes the request and sends back a response message to the requesting client. In order to improve performance, we will create a proxy server between the client and the original web server. Now, both the request message sent by the client and the response message delivered by the original web server pass through the proxy server. In other words, the client requests the objects via the proxy server. The proxy server will forward the client's request to the original web server. The original web server will then generate a response message and deliver it to the proxy server, which in turn sends it to the client.



Getting started: socket programming review in Python 3 or Python 2

In this assignment, you will use Python 3's or Python 2's socket API to implement the TCP connections used by the proxy server. **Python2 is recommended if you don't want to modify/debug too much for the skeleton code at the end.** To get started with Python socket programming, please review the TCP client/server examples presented at the following link:

[TCP/IP Client and Server - PyMOTW-3](#) (Ctrl+Click to open the link)

Below is the link to the official documentation of the Python 3 socket library:

[socket - Low-level networking interface - Python 3.11.1 documentation](#) (Ctrl+Click to open the link)

Once you are caught up with socket programming, you may start completing your tasks in this assignment.

Code for You to Start With

At the end of this document, you will find the skeleton code for the project (*note: the skeleton has bugs that you need to fix*). You are to complete the skeleton code. The places where you need to fill in code are marked with **#Fill in start** and **#Fill in end**. Each place may require several lines of code. **Beside the “fill in” places, your team also needs to figure out how to solve the bugs in the skeleton code as well (Hint: if you use Python 2, it would be easier to finish the project since the skeleton itself is written in Python2).**

Things to Pay Attention

Make sure your browser program is not, by default, using the HTTP/3 protocol or the QUIC protocol.

We've learned about HTTP/3 and QUIC. You should know that when you're running HTTP/3 or QUIC, upperlayer protocol information (HTTP, TCP) sent by your computer will be encrypted. As of 2025, most popular browsers have adopted HTTP/3 and QUIC as defaults. A document that describes how to disable HTTP/3 AND QUIC as the defaults for your browser is here: <https://techysnoop.com/disable-quic-protocol-in-chrome-edge-firefox/>

Please note that if you don't disable HTTP/3 protocol or the QUIC protocol, you might not be able to visit the web pages I provided in the project!

Make sure then when you go to a URL, that the URL starts with "http://" rather than with https:// as the latter case will encrypt frames.

Part 1: Testing the Proxy Server on One Computer

Run the proxy server program using your command prompt and then request a web page from your browser. Direct the requests to the proxy server using your IP address and port number.

For e.g., `http://localhost:8888/gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html`

You don't need to submit anything for the part1.

Part2: Testing the Proxy Server on Two Computers

To use the proxy server technique with browser and proxy on separate computers, you will need the IP address on which your proxy server is running (Note: it should be running on one team member's computer). In this case, while running the proxy, you will have to replace the "localhost" with the IP address of the computer where the proxy server is running. Also note the port number used. You will replace the port number used here "8888" with the port number you have used in your server code at which your proxy server is listening. On the other team member's computer (i.e., the one that's not running proxy server), open a browser and enter the webpage that you want to cache.

For e.g.,

`http://<IP of the proxy server>:<port number of the proxy server>/gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html`

***** When you test your proxy server with your teammate, your computers should connect to the same network (i.e., you are in the same LAN) *****

*** I noticed that some teams' project 2 in the previous semester works very well on one computer, however, doesn't work on two computers. ***

Two potential solutions are as follows:

1) Disable Windows/Mac/Linux Firewall Temporarily on Both Computers

- Temporarily disable the Firewall and any Antivirus software (AV) programs and see if you can then connect. If so, you know the problem is with Firewall or your AV program blocking connections.

2) If disabling Firewall doesn't work, you may also try to let your current "server" computer run the client code and let your current "client" computer run the server code. I found it helps solve the issue of some teams.

Enable caching

For each requested URL, we save their response from the destination in a file on the disk (so the cache persists when we terminate and restart the proxy server). Next time the same URL gets requests, we will load the response from the corresponding file on the disk rather than creating a connection to the destination server. You'll see that this greatly improves the page loading speed on the browser side.

Things to think about in this step: How do we name the cache files, i.e., how do we convert the URLs into proper filenames? Things not to worry about: you don't need to worry about replacement policies, i.e., how to evict an item out of the cache when the cache becomes "full". We simply assume that we have enough disk space, and we never need to evict any item.

Note: To make your program portable, the cache files should be located under the current folder (where **teamXproxy.py** is).

Configuring your Browser

You can also directly configure your web browser on the client PC to use the proxy server as default. This depends on your browser. In Internet Explorer on the client, you can set the proxy in Tools > Internet Options > Connections tab > LAN Settings. In Netscape (and derived browsers such as Mozilla) on the client, you can set the proxy in Tools > Options > Advanced tab > Network tab > Connection Settings. In both cases you need to give the address of the proxy server and the port number that you gave when you ran the proxy server. With this approach, to get a web page using the proxy server, you simply provide the URL of the page in the client's web browser **WITHOUT entering <IP of the proxy server>:<port number of the proxy server> after http**. The following is an example of URL that you can enter in the client's web browser after configuration.

<http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html>

How to Run and Test Your Proxy Server

- On one team member's computer (**PC 1**), start the proxy by running the following command in Terminal/Command Prompt: **python3 teamXproxy.py <IP address of PC 1>** (**note: X in teamXproxy.py file is your team number**)
- After that, on the other team member's computer (**PC 2**), configure the browser appropriately as described above. Predetermine **three different webpages** to visit and store the webpage links info into a report file (e.g., **teamX_proxy_performance_analysis.docx**).
- Please first review "Wireshark_Lab1_intro.pdf" in Optional Assignment – Lab 1 on Canvas for instructions on how to use Wireshark. By reading the document and testing the procedures described, you should be able to figure out how to measure the webpage retrieval time.
- **Clear the browser's cache** (*start up your web browser, and make sure your browser's cache is cleared. Please refer to the following link to delete the browser's cache:* <https://www.howtogeek.com/304218/how-to-clear-your-history-in-any-browser>), then open Wireshark and start capturing. After that, open three predetermined webpages respectively. Stop Wireshark capturing. Check how long it takes till the http response is received for each webpage and log that info into the report file. And you should also find that under the folder where you save teamXproxy.py file, **three different files (i.e., caches)** are generated.
- **Close the browser, wait for at least one minute, reopen the browser, and clear the browser's cache** (*note: reopening the browser typically clears its memory cache, but clearing it manually provides an extra guarantee*). Then restart Wireshark capturing, and visit those three webpages respectively again. Stop Wireshark capturing and check how long it takes to fetch each webpage this time. Log that info into the report as well.

- For writing the report file, the format could be similar to the following figure:

Web Page	Response Time without Proxy Server	Response time with Proxy Server

- To ease your job on project 2, you don't need to handle HTTPs and HTTP 2.0 protocol, which means you DON't need to handle websites such as <https://www.google.com/>. Instead, please only test webpages that are using HTTP 1.1.
 - Please go check the webpages you tested in Wireshark lab 1 (HTTP protocol), such as:
 1. <http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html>
 2. <http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html>
 3. <http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html>
 - Those webpages are using HTTP 1.1 protocol, and you use those webpages to test your proxy server implementation.

The Things that're **NOT** Required

- Your proxy server does not need to handle other requests except GET requests.
- You do not need to implement any replacement or validation policies. Check the following text to refresh your knowledge.
 - Based on the description of this project, the proxy works as follows: When the proxy server gets a request, it checks if the requested object is cached, and if yes, it returns the object from the cache, without contacting the original web server. If the object is not cached, the proxy retrieves the object from the server, returns it to the client and caches a copy for future requests. **In practice, the proxy server must verify that the cached responses are still valid and that they are the correct responses to the client's requests (e.g., If-Modified-Since).** You can read more about caching and how it is handled in HTTP in RFC 2068.
 - **In this project, you DO NOT need to implement any replacement or validation policies, which means you just fetch whatever is cached in the proxy server.**

What to Hand in

You will hand in the complete proxy server code (**teamXproxy.py**) and **screenshots** at the client side verifying that you indeed get the web page via the proxy server. And a report file (i.e., **teamX_proxy_performance_analysis.docx**) as described above. And **three caches file** that your proxy generates for you when you visit the three webpages. Both you and your teammate need to **zip** the following files (including **teamXproxy.py**, **screenshots**, **teamX_proxy_performance_analysis.docx**, **three caches file**) into one file called **teamX_report.zip** to Canvas and make sure you and your teammate upload the same version since the grader will only check one.

Programming Language

A note on programming languages:

- You are encouraged to write your proxy server in Python 2 or Python 3 since that's the language used in the lecture for sockets. But you can also write it in Java or C if you prefer.

Skeleton Python Code for the Proxy Server (*there are bugs in the code!*)

```
from socket import *

import sys

if len(sys.argv) <= 1:

    print('Usage : "python ProxyServer.py server_ip"\n[server_ip : It is the IP Address Of Proxy Server]')

    sys.exit(2)

# Create a server socket, bind it to a port and start listening

tcpSerSock = socket(AF_INET, SOCK_STREAM)

# Fill in start.

# Fill in end.

while 1:

    # Start receiving data from the client

    print('Ready to serve...')

    tcpCliSock, addr = tcpSerSock.accept()

    print('Received a connection from:', addr)

    message = # Fill in start.          # Fill in end.

    print(message)

    # Extract the filename from the given message

    print(message.split()[1])

    filename = message.split()[1].partition("/")[2]

    print(filename)

    fileExist = "false"

    filetouse = "/" + filename

    print(filetouse)

    try:

        # Check whether the file exist in the cache

        f = open(filetouse[1:], "r")
```

```

outputdata = f.readlines()

fileExist = "true"

# ProxyServer finds a cache hit and generates a response message

tcpCliSock.send("HTTP/1.0 200 OK\r\n")

tcpCliSock.send("Content-Type:text/html\r\n")

# Fill in start.

# Fill in end.

        print('Read from cache')

# Error handling for file not found in cache

except IOError:

    if fileExist == "false":

        # Create a socket on the proxyserver

        c = # Fill in start.                # Fill in end.

        hostn = filename.replace("www.", "", 1)

        print(hostn)

        try:

            # Connect to the socket to port 80

            # Fill in start.

            # Fill in end.

            # Create a temporary file on this socket and ask port 80
for the file requested by the client

            fileobj = c.makefile('r', 0)

            fileobj.write("GET      "+"http://"      +      filename      +      "
HTTP/1.0\n\n")

            # Read the response into buffer

            # Fill in start.

            # Fill in end.

            # Create a new file in the cache for the requested file.

            # Also send the response in the buffer to client socket
and the corresponding file in the cache

            tmpFile = open("./" + filename, "wb")

```

```
        # Fill in start.

        # Fill in end.

    except:

        print("Illegal request")

    else:

        # HTTP response message for file not found

        # Fill in start.

        # Fill in end.

    # Close the client and the server sockets

    tcpCliSock.close()

# Fill in start.

# Fill in end.
```